

ARQUITECTURA DE SISTEMA BANCARIO

Alcance del sistema

El sistema bancario permitirá que los clientes del banco puedan revisar el histórico de movimientos de sus cuentas, realizar pagos y transferencias a cuentas propias, así como también a otros bancos y recibir las notificaciones de los movimientos realizados vía SMS y correo electrónico.

Los clientes del banco podrán hacer uso del sistema a través de una aplicación móvil y/o de una página web, las cuales permitirán al usuario realizar consultas de movimientos, pagos y transferencias. Todo esto será posible siempre que el cliente se haya registrado en el sistema bancario y haya pasado las validaciones de autorización y autenticación. La aplicación móvil también usará el reconocimiento facial como alternativa para acceder al sistema bancario.

El sistema cumplirá con los estándares de seguridad de la industria para proteger la información de los clientes y las diferentes interacciones que puede realizar con sus cuentas bancarias o productos. Esto incluye el uso de herramientas de detección de fraude, que pueden ser usadas en los flujos más críticos como: enrolamiento o registro en el sistema, pagos y transferencias.

El sistema tendrá la capacidad de recopilar y administrar los datos necesarios para mantener auditoría de todas las acciones que hagan los clientes del banco cuando usen el sistema. En ningún momento esta capacidad degradará el rendimiento del sistema.

La implementación del sistema debe cumplir con las normas legales del país en el cual se encuentra establecido el banco, sin perjuicio de aplicar normas y estándares internacionales como:

- Ley de protección de datos (Según norma legal del país)
- ISO-27001
- PCI-DSS
- Zero Trust
- etc

Objetivos y restricciones de arquitectura

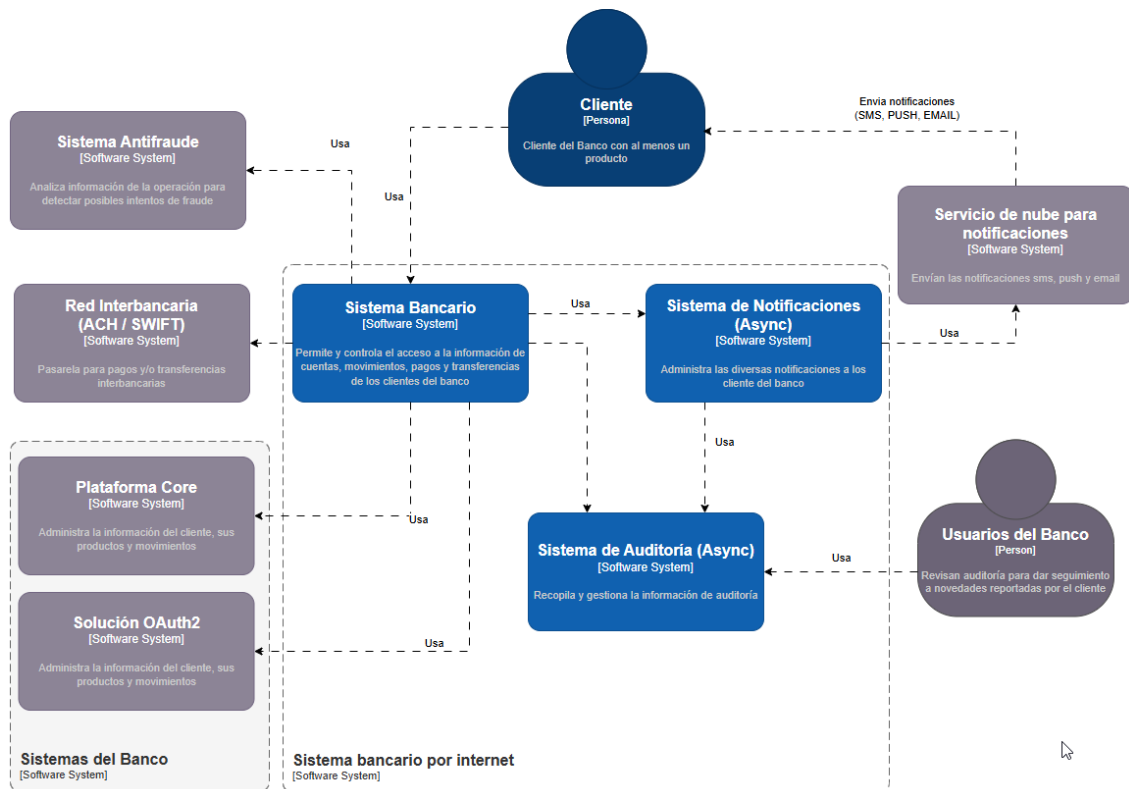
La arquitectura propuesta para el sistema bancario busca los siguientes objetivos:

- Alta disponibilidad
- Escalabilidad
- Bajo acoplamiento
- Tolerancia a fallos
- Recuperación de desastres
- Seguridad y monitoreo
- Excelencia operativa
- Auto-healing
- Reusabilidad y cohesión de componentes

Diagramas

C1- Diagrama de Contexto del Sistema Bancario

Este diagrama muestra como un cliente (usuario del sistema) utiliza el sistema bancario para acceder a su información, productos y movimientos y como el sistema bancario interactúa con los sistemas de notificaciones y de auditoría. Contempla la existencia de un usuario del banco que revisa y analiza la información de la auditoría para dar seguimiento a posibles novedades reportadas por los clientes que hacen uso del sistema bancario. Muestra también la interacción con el sistema antifraude y la red interbancaria.



Usuarios

Cliente: Usuario final registrado en el sistema bancario, el cual accede al sistema vía web o app, para realizar las transacciones económicas (pagos y/o transferencias) y consultas de movimientos.

Usuario del Banco: Usuario del banco que tiene acceso a datos de auditoría para monitorear o dar seguimiento a novedades reportadas por el cliente.

Sistemas Internos

Sistema Bancario: Brinda las facilidades, validaciones y controles para que un usuario pueda revisar los movimientos de sus cuentas, realizar pagos y/o transferencias de forma segura.

Sistema de Notificaciones: Permite la configuración, administración, preparación y envío de las diferentes notificaciones que el sistema bancario debe hacer llegar a los usuarios (sms, push y correo electrónico); como resultado de una transacción monetaria (pago o transferencia) o como parte de un flujo específico (enrolamiento o login) el cual requiera enviar notificaciones al cliente.

Sistema de Auditoría: Recopila, procesa, almacena y gestiona los datos necesarios para mantener auditoría de todas las acciones que realiza el usuario en el sistema bancario, de tal forma que puedan estar disponibles a los usuarios autorizados del banco para que puedan dar seguimiento a novedades reportadas por el cliente en el uso del sistema bancario.

Sistemas Externos

Sistema Antifraude: Analiza la información de cada operación para detectar posibles intentos de fraude.

Red Interbancaria: Se encarga de recibir y direccionar las transacciones a cuentas de otros bancos en el mismo país o cuentas en bancos de otros países.

Plataforma Core (On-Premise): Administra la información básica de los clientes del banco, sus productos y movimientos. Este sistema debe estar On-Premise para cumplir con las normas legales que generalmente rigen a los bancos.

Solución OAuth2: Producto de Software que tiene el banco para la autenticación y autorización de usuarios.

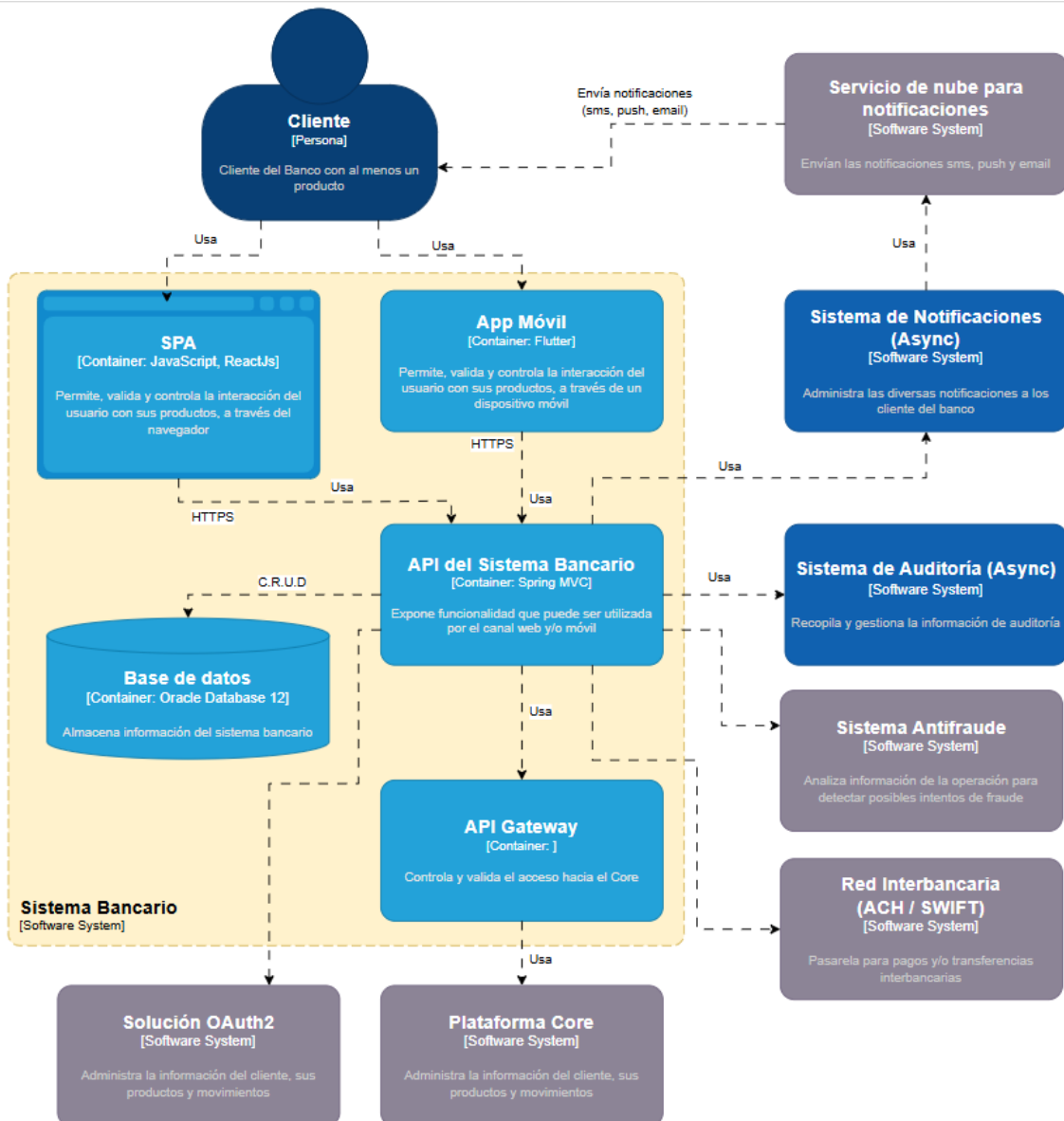
Consideraciones del Diseño

- La Plataforma Core debe mantenerse On-Premise para cumplir normativas que rigen a los bancos para mantener los datos críticos en infraestructura que esté bajo el completo control del banco.
- Se propone una arquitectura híbrida:
 - On-Premise: Para mantener los datos críticos y sensibles en infraestructura controlada por el banco.
 - Cloud para lograr la escalabilidad (y flexibilidad) requerida(s) ante el posible incremento de tráfico al liberar los canales: móvil (App) y web (SPA); bajo un esquema “controlado” de pago por uso de recursos de la nube.
- Se plantea un sistema asíncrono de notificaciones (sms, push y email) para garantizar la reusabilidad de su funcionalidad, bajo acoplamiento de los componentes y también para cubrir toda la complejidad (plantillas, redundancia de canales, integración con proveedores, etc) que pueden llegar a tener cada una de las notificaciones necesarias en los sistemas bancarios.
- Se plantea un sistema de auditoría que será asíncrono para mantener el alto desempeño esperado de un sistema bancario el cual también permitirá que usuarios del banco puedan acceder a dicha información para validar las acciones realizadas por los usuarios en el sistema bancario.

C1- Diagramas de Contenedores

C1-01 Diagrama de Contenedores - Sistema Bancario

El diagrama muestra como el cliente usa la SPA y/o la Aplicación Móvil para acceder a la funcionalidad que ofrece el sistema bancario a través de su API. Se aprecia dónde se almacenan los datos del sistema bancario y cómo utiliza el API-GATEWAY para obtener información de CORE.



Contenedores:

Single Page Application (SPA): Interfaz implementada en un navegador web para que el usuario pueda realizar consultas de movimientos, pagos y transferencias. Aquí se aplican las validaciones necesarias a la información que es ingresada por el usuario durante su interacción con el sistema.

App Móvil: Interfaz implementada para que el usuario pueda interactuar con el sistema, a través de un dispositivo móvil. Esta implementa validaciones a los datos ingresados por el usuario durante su interacción con el sistema. Si el dispositivo móvil lo permite, la App móvil facilitará el reconocimiento facial y/o huella dactilar (datos biométricos) para ser utilizados en los procesos de autorización y autenticación del usuario en el sistema.

API del sistema bancario: Expone todas las funcionalidades que pueden ser consumidas por la página web (SPA) o la aplicación móvil, para que el usuario pueda interactuar con sus cuentas, haciendo consultas de movimientos, pagos y/o transferencias. Esta API permite mantener un acoplamiento débil con la capa de presentación (SPA y App).

API Gateway: Permite, controla y facilita el acceso a la información que se administra en el CORE.

Base de datos: Almacena la información del sistema garantizando que esté disponible en cualquier momento para ser mostrada al usuario.

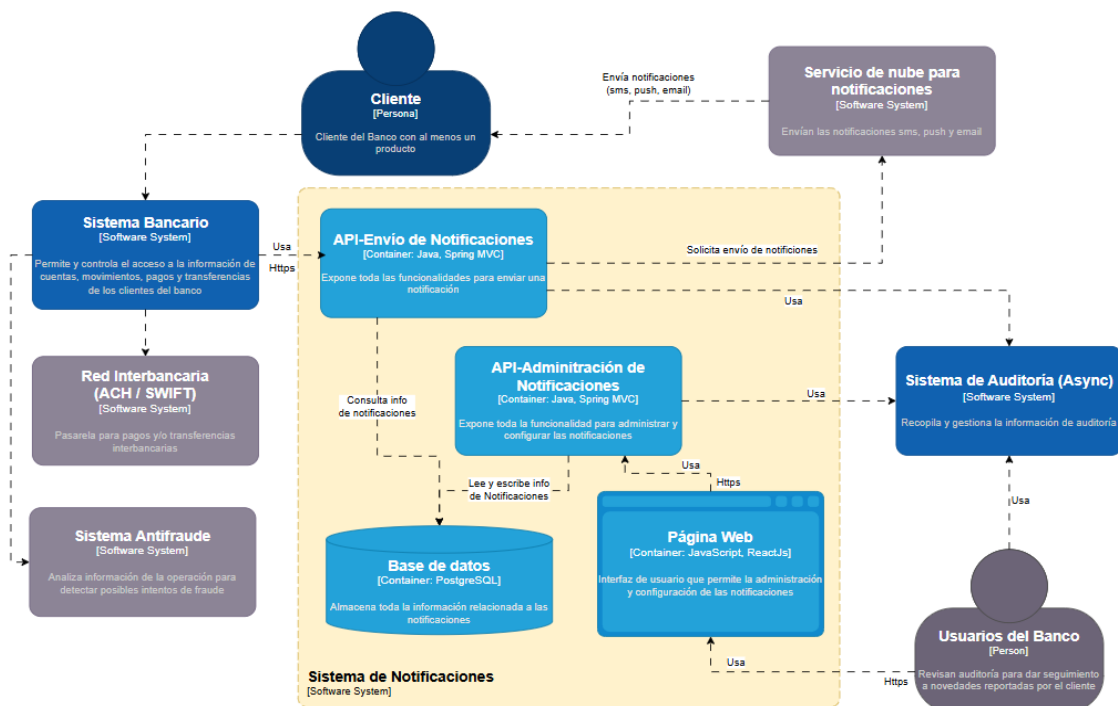
Solución OAuth2: Producto de Software que tiene el banco para la autenticación y autorización de usuarios.

Consideraciones del Diseño

- Las conexiones desde la capa de presentación (SPA y App) hacia la API del sistema bancario se harán a través de HTTPS con TLS 1.3, para mantener segura la información en tránsito.
- La aplicación web (SPA) debe ser responsive y debe implementar mecanismos de protección contra XSS e inyección de SQL
- Se sugiere realizar la aplicación móvil utilizando Flutter por el gran soporte que brinda la comunidad. Se consideró otras plataformas como Outsystems pero ésta podría ser prohibitiva por los costos de sus licencias y los SLAs asociados a cada licencia.
- El uso de la Flutter para crear la App para Android como para iOS permite:
 - o Reducir el tiempo y costo de desarrollo
 - o Mejora la mantenibilidad de la App para las dos plataformas
 - o Garantiza la misma experiencia de usuario e iOS y Android
- La capa de presentación (SPA y App) estará desacoplada del backend a través del API del sistema bancario

C2-02 Diagrama de Contenedores - Sistema de Notificaciones

El diagrama muestra como el sistema bancario utiliza la API de envío de notificaciones para enviar una notificación (sms, push y email) al cliente. Para hacerlo la API de envío de notificaciones consulta la base de datos para obtener la plantilla y configuración de la notificación solicitada y luego solicita el envío de dicha notificación al servicio de nube para notificaciones el cual enviará un sms, push y/o email, según corresponda.



Contenedores

API-Envío de notificaciones: Se encarga de consultar, preparar y enviar la notificación que el sistema bancario haya solicitado. Esta API puede ser utilizada por otros componentes de la solución, permitiendo así la reutilización de las capacidades del sistema de notificaciones.

API-Administración de Notificaciones: Se encarga de exponer toda la funcionalidad necesaria para administrar y configurar las notificaciones de toda la solución. Esta API es utilizada por la página web a la cual tienen acceso los usuarios del banco

Página Web: Interfaz para los usuarios del banco que brinda las facilidades para administrar y configurar las notificaciones del sistema bancario, así como de otros componentes (actuales y futuros) de la solución.

Base de datos: Almacena la información relacionada con las notificaciones y la mantiene disponible para que sea gestionada por los usuarios del banco.

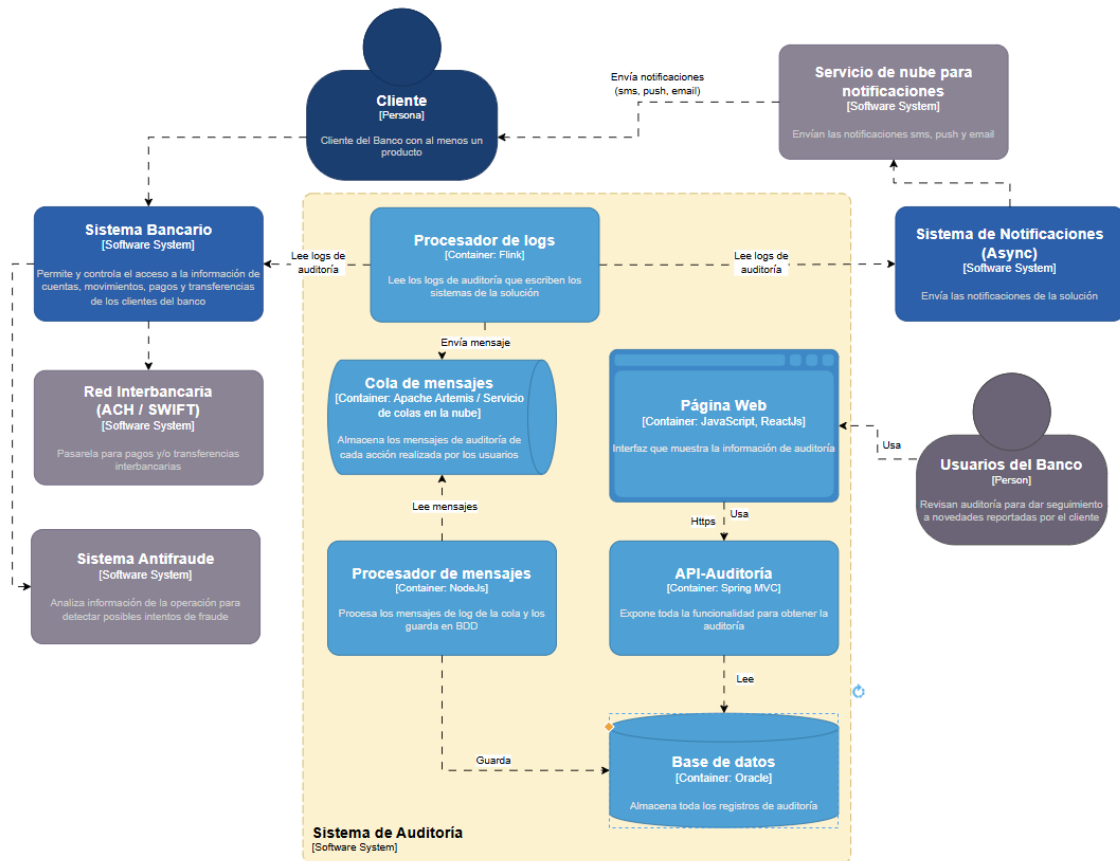
Servicio de nube para notificaciones: Se encarga de realizar la entrega de las notificaciones que solicita el sistema bancario (u otro sistema de la solución) a través del sistema de notificaciones. Estas notificaciones se entregan como sms, push (para los móviles) y email, según corresponda.

Consideraciones del Diseño

- Las conexiones hacia las APIs del sistema de notificaciones se realizarán por HTTPS con TLS 1.3
- Las APIs se implementarán con Spring MVC
- La base de datos a utilizar será PostgreSQL v17. Se consideró la base de datos Oracle 12 pero se descartó por tener un costo de licenciamiento superior al de PostgreSQL.
- La funcionalidad del sistema de notificaciones puede ser reutilizado por otros sistemas de la solución.
- El servicio de nube para las notificaciones deberá permitir el envío de notificaciones por SMS, PUSH en caso de la App y por email.
- Tanto la administración de notificaciones, como el envío de estas quedará registrado en el sistema de auditoría

C2-03 Diagrama de Contenedores - Sistema de Auditoría

Este diagrama muestra como el sistema de auditoría lee los logs de auditoría que escriben los otros sistemas de la solución (bancario, notificaciones), para luego procesar dichos registros y almacenarlos en base de datos para que puedan ser consultados por los usuarios del banco para dar seguimiento a las novedades reportadas por los clientes.



Contenedores

Procesador de logs: Se encarga de leer los logs de auditoría que escriben los sistemas de banco y de notificaciones; y de enviarlo a la cola de mensajes para ser procesados. Estas lecturas se hacen de forma asíncrona cada vez que roten los logs lo cual garantiza mantener bajo acoplamiento de los componentes de la solución y sin degradar el rendimiento de esta.

Cola de mensajes: Almacena los mensajes de auditoría para que sean procesados una sola vez por el "Procesador de mensajes".

Procesador de mensajes: Lee los mensajes de auditoría de la cola de mensajes, los procesa, formatea, etc y persiste en la base de datos.

Base de datos: Almacena la información de auditoría y la mantiene disponible para que pueda ser visualizada por los usuarios del banco.

API-Auditoría: Expone toda la funcionalidad necesaria para que la información de auditoría pueda ser visualizada de forma rápida y segura, llegando a auditar también las acciones de los usuarios del banco.

Página web: Interfaz que brinda las facilidades para que el usuario de banco pueda visualizar la información de auditoría para dar seguimiento a novedades reportadas por el cliente.

Consideraciones del Diseño

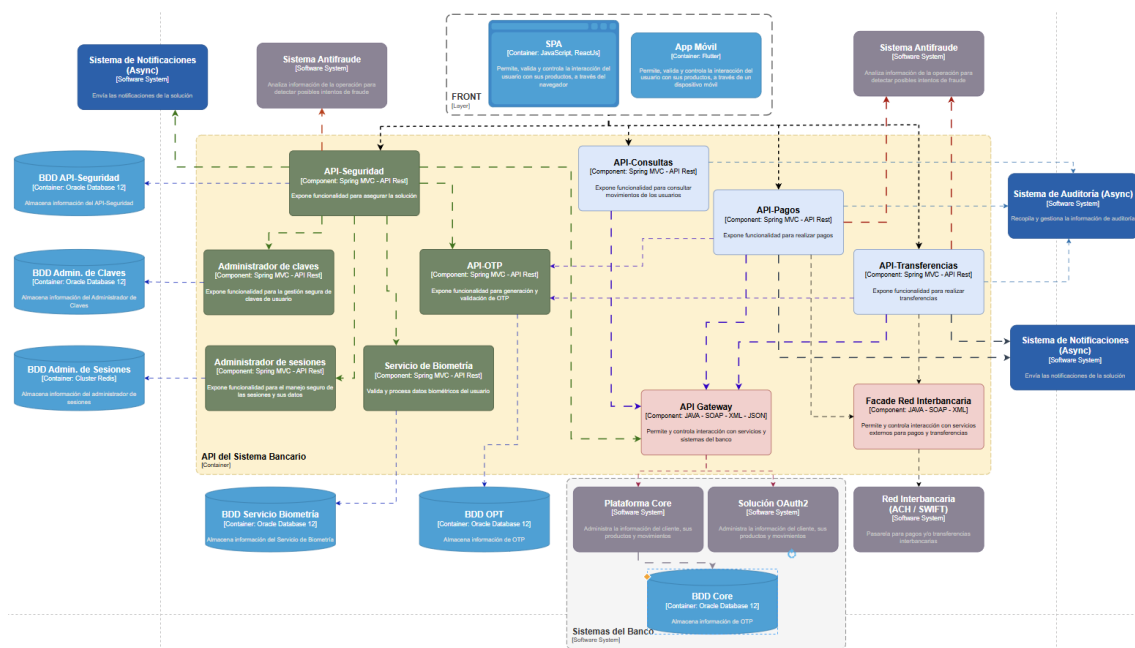
- Los componentes están bajamente acoplados

- La funcionalidad puede ser fácilmente configurada para leer y procesar logs de auditoría de otros sistemas de la solución, permitiendo así reusabilidad de esta funcionalidad de la solución.
- Se mantiene el protocolo HTTPS con TLS 1.3 para las comunicaciones con la API de auditoría
- La auditoría se implementará con los patrones WORM (Write once – Read many) y Event Sourcing para habilitar procesos de recuperación ante desastres.
- La información de auditoría debe retenerse por el tiempo que exija la ley del país donde se encuentra el banco
- El acceso a la auditoría será restringido siguiendo el principio “Zero Trust”

C3- Diagramas de Componentes

C3-01-01 Diagramas de Componentes – API del Sistema Bancario

El diagrama muestra cómo interactúan los componentes del API del sistema bancario para que el usuario pueda realizar consultas de movimientos, pagos y transferencias. Se utilizará el patrón de microservicios para implementar los servicios del sistema Bancario.



Componentes:

API-Seguridad: Expone toda la funcionalidad necesaria para garantizar un acceso seguro a los diferentes recursos del sistema bancario. Este hace uso de la funcionalidad de otros componentes especializados para lograr su objetivo de mantener segura la solución y protegerlos los pagos y transferencias usando flujos seguros.

Administrador de claves: Expone la funcionalidad necesaria para gestionar las claves de usuario de la solución. Se encarga de almacenar de forma segura el hash de un password, validar su vigencia, fortaleza, reseteo de clave, etc.

API-OTP: Expone la funcionalidad necesaria para generar y validar los OTPs que son requeridos como segundo factor de autenticación (MFA) durante varios flujos de la solución, como pagos, transferencias, reseteo de claves, enrolamiento, etc.

Administrador de sesiones: Se encarga de administrar las sesiones de la solución de forma segura, así como de la información que se almacena en cada una de ellas. Controla el tiempo de vida de la sesión, permite iniciar, mantener y cerrar las sesiones.

Servicio de Biometría: Se encarga de almacenar y validar la información biométrica de los usuarios del sistema bancario.

API-Consultas: Expone todos los servicios necesarios para que la APP o la SPA, puedan acceder a las consultas relacionadas con los productos que el cliente tiene con el banco, históricos de movimientos, etc.

API-Pagos: Expone la funcionalidad necesaria para que un usuario pueda realizar pagos de forma segura a través de la solución.

API-Transferencias: Expone la funcionalidad necesaria para que el usuario pueda realizar transferencias entre sus cuentas o transferencia a cuentas de otros bancos o de terceros

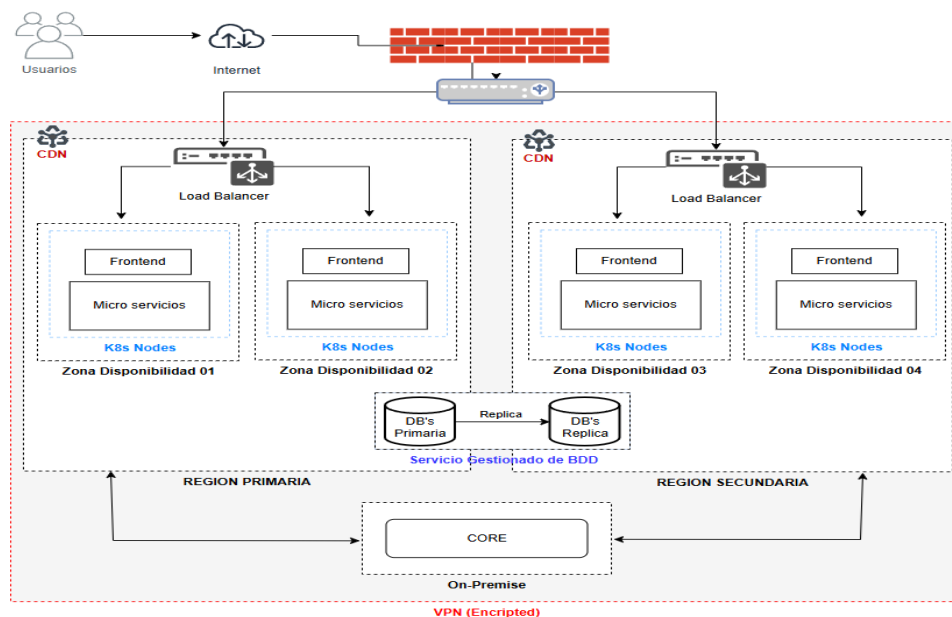
API Gateway: Permite una interacción desacoplada con la plataforma del Banco para acceder a la información que es gestionada por el componente “Plataforma core”

Facade Red Interbancaria: Encapsula los servicios necesarios para enviar peticiones de transferencias internacionales o interbancarias a la red Swift o ACH del país en el cual se encuentra el Banco.

Despliegue en nube

Diagrama de Despliegue de alto nivel

El diagrama muestra como interactúa el CORE alojado On-Premise con los demás componentes de la solución que se encuentran en la nube.



Flujos Destacados

Flujo de Onboarding desde Aplicación Móvil

Este flujo describe el proceso que debe seguir un cliente del banco para registrarse en el sistema bancario. Para este flujo se asume lo siguiente:

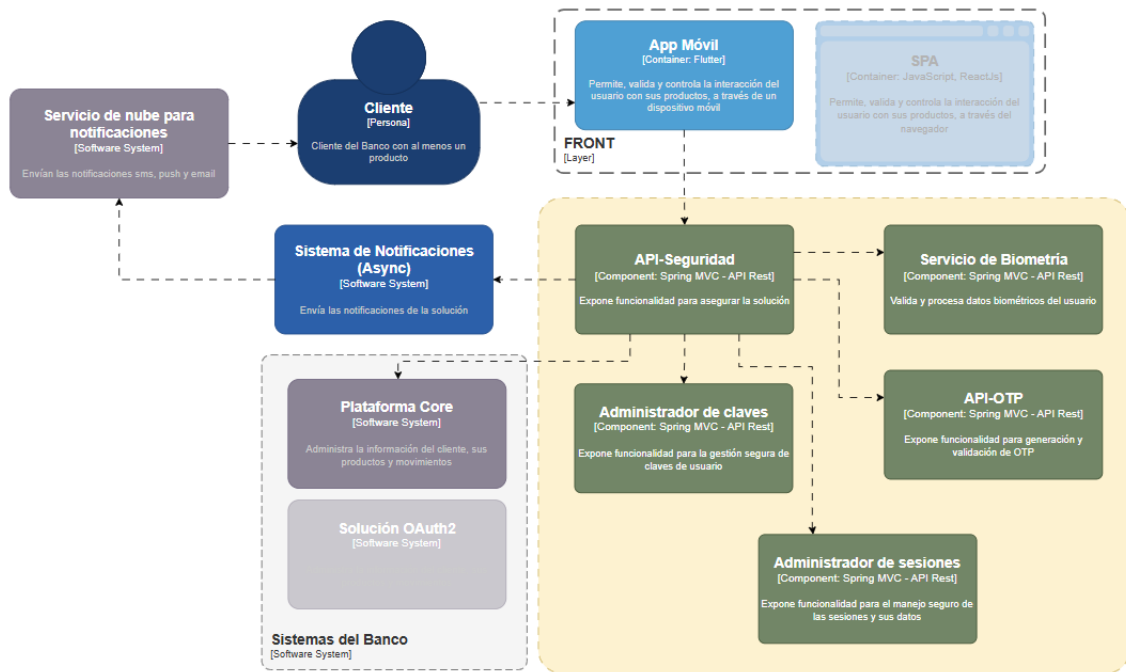
- La aplicación móvil está instalada en el dispositivo del cliente
- La aplicación móvil tiene conexión segura hacia el backend
- La aplicación móvil tiene acceso a la cámara frontal del dispositivo
- El flujo es de alto nivel, es el “camino feliz” (happy path) y no produce errores
- Los sub-flujos posibles son muchos y no se detallan a continuación

Actor	Actividad
Usuario	Abre la aplicación instalada en su dispositivo móvil e ingresa al proceso de registro
API-Seguridad	Utiliza el “Administrador de sesiones” para crear una nueva sesión
App	Verifica que el dispositivo tenga acceso a la cámara y la capacidad de reconocimiento facial
App	Muestra instrucciones de cómo el usuario debe tomarse la foto para obtener sus datos biométricos
App	Habilita la cámara frontal para que el usuario pueda tomarse la
Usuario	Toma la foto con la cámara frontal
App	Envía la foto de forma cifrada hacia el backend
API-Seguridad	Recibe la foto y hace uso del “Servicio de biometría” para obtener los datos biométricos
Servicio de Biometría	Analiza la foto y obtiene los datos biométricos de la misma
API-Seguridad	Dirige el flujo para que la APP solicite una foto del documento de identidad del cliente que está registrándose en el sistema bancario
App	Solicita al usuario que tomen una foto de su documento de identidad
Usuario	Toma la foto del documento y la envía
App	Envía la foto del documento en forma cifrada hacia el backend
API-Seguridad	Recibe la imagen y la envía al “Servicio de biometría” para que la analice y compare con la imagen del rostro del cliente que se envió anteriormente
Servicio de Biometría	Analiza y compara la imagen del rostro del cliente que se envió anteriormente. El servicio de biometría responderá: - Los rostros de la identificación y del usuario coinciden - Los rostros de la identificación y del usuario NO coinciden - El riesgo de que pueda ser un intento de fraude ***Para este flujo se supondrá que los rostros coinciden***
API-Seguridad	Recibe la respuesta “exitosa” del servicio de biometría y dirige el flujo para que se solicite más datos del usuario
App	Muestra controles para que el usuario ingrese su correo electrónico
Usuario	Ingresa su correo electrónico y presiona enviar
App	Encripta el correo electrónico
API-Seguridad	Consume la API-OTP para generar un OTP y hace uso del API-Notificaciones para enviar dicho OTP al email ingresado para confirmar que el usuario es dueño de la cuenta de correo electrónico

App	Muestra controles para que el usuario ingrese el OTP enviado a su email
Usuario	Ingresa el código que recibió en su email y lo envía
App	Encripta y envía el OTP ingresado por el usuario
API-Seguridad	Recibe el OTP ingresado por el usuario y usa la API-OTP para validar el código recibido
API-OTP	Valida y confirma que es correcto
API-Seguridad	Hace uso de la “Plataforma Core” del banco para crear el usuario con datos iniciales que permitirán retomar de forma segura; el registro del usuario, en caso de que éste se interrumpa. Los datos son: - ID único del usuario en la “Plataforma Core” del banco - Correo electrónico (ya verificado con la validación del OTP) - Biometría del reconocimiento facial Dirige el flujo para que la App solicite más datos del usuario
App	Muestra controles para que el usuario ingrese y envíe encriptada la siguiente información personal: - Nombres - Apellidos - Fecha de nacimiento - Sexo - Dirección del domicilio
API-Seguridad	Recibe y valida la información enviada para luego hacer uso de la “Plataforma Core” para guardar dicha información del usuario. Dirige el flujo para que la App solicite más datos del usuario
App	Muestra una lista seleccionable de productos para que el usuario indique cuál desea: - Cuenta de Ahorros - Cuenta Corriente
Usuario	Selecciona el producto que desea y presiona enviar
API-Seguridad	Recibe y valida la información enviada para luego hacer uso de la “Plataforma Core” para generar y registrar el número que identifique al producto seleccionado. Este número de producto (Número de cuenta) pertenece solo al usuario que se está registrando. Dirige el flujo para que la App solicite más datos del usuario
App	Muestra controles para que el usuario ingrese y envíe encriptados: el usuario y la clave. Solicita al usuario que digite dos veces la misma clave para confirmarla.
Usuario	Ingresa un usuario y su clave (digitada dos veces para confirmarla)
App	Encripta la información de usuario y clave y la envía al backend
API-Seguridad	Recibe y valida la información enviada. Guarda el nombre de usuario y haciendo uso del “Administrador de claves” valida y almacena el hash de la contraseña ingresada por el usuario.
App-Seguridad	Consume los servicios del “Administrador de sesiones” para invalidar y cerrar la sesión con la cual se realizó exitosamente el registro del usuario. Dirige el flujo para que la App muestre un mensaje de éxito
App	Muestra mensaje indicando que el usuario se registró exitosamente.

Componentes de la Arquitectura de Onboarding

El siguiente diagrama muestra los componentes que permiten el proceso de onboarding de un nuevo cliente del banco.



Flujo de Login desde Aplicación móvil

Este flujo describe el proceso que debe seguir un usuario del Banco para ingresar en el sistema haciendo uso de la aplicación móvil.

Para este flujo se asume lo siguiente:

- El cliente siguió el “Flujo de Onboarding” para registrarse en el sistema
- La aplicación móvil está instalada en el dispositivo del cliente
- La aplicación móvil tiene conexión segura hacia el backend
- La aplicación móvil tiene acceso a la cámara frontal del dispositivo
- El flujo es de alto nivel, es el “camino feliz” (happy path) y no produce errores
- La aplicación móvil reconoce “URIs Personalizadas” (Custom URI Scheme)
- Los sub-flujos posibles son muchos y no se detallan a continuación

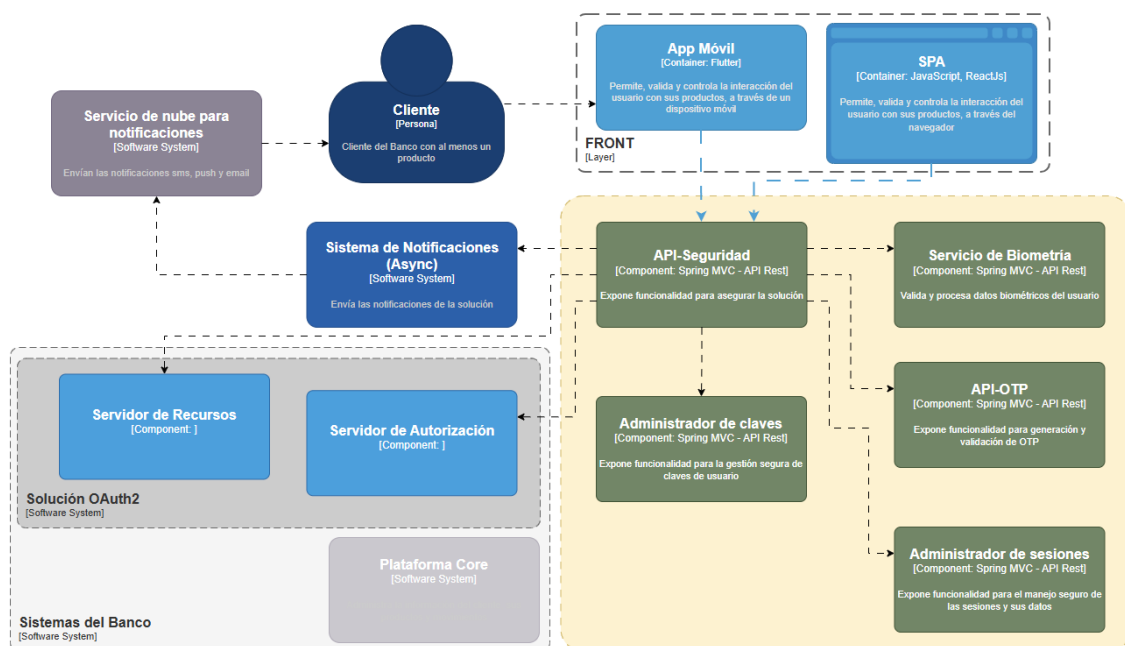
Actor	Actividad
Usuario	Abre la App
App	Genera un código de verificación (code verifier) y su respectivo hash (code challenge) y abre una pestaña segura del navegador apuntando al Servicio de OAuth2 que tiene el banco, para que el usuario introduzca sus credenciales. Como parte del request se envía el “code challenge” del código de verificación.
Servidor de Autenticación	Recibe la petición con el “code challenge” y la URL hacia la cual redirigir al usuario luego de validar exitosamente sus credenciales. Almacena el “Code Challenge” para futura validación Muestra controles para que el usuario ingrese su usuario y clave o haga su validación de identidad por reconocimiento facial.
Usuario	Ingresa su usuario y clave en el sitio seguro de OAuth2 del Banco o utiliza el reconocimiento facial

Servidor de Autorización	Verifica las credenciales ingresadas por el usuario o sus datos biométricos del reconocimiento facial. En función del análisis de riesgo de esta acción se podría llegar a solicitar un segundo factor de autenticación como OTP.
Servidor de Autorización	Solicita la confirmación del usuario para que se pueda acceder a su información
Usuario	Aprueba los permisos que fueron solicitados, siguiendo siempre el principio de “mínimos privilegios”
Servidor de Autorización	Redirige el flujo hacia la URI de redirección de la App enviando el código de autorización
App	Detecta la URI Personalizada y recibe el código de autorización
App	Hace una petición (POST) al servidor de autorización enviando el código de autorización y el código de verificación (que calculó al inicio del flujo), el cual solo lo conoce la App y nunca antes fue enviado
Servidor de Autorización	Calcula el hash del código de verificación que recibió en la petición y lo compara con el “code challenge” que se envió al inicio del flujo. También valida el código de autorización. Si las dos validaciones son correctas retorna los tokens de acceso y de refresco
App	Almacena el token de refresco en el almacén seguro del dispositivo móvil y utiliza el token de acceso para hacer las llamadas al API del sistema bancario.

Considerando que la App y la SPA son “clientes públicos” que pueden ser vulnerados y que intentan acceder a la información del cliente protegida por el banco, el flujo de login debe implementarse considerando PKCE (Proof Key Code Exchange), tal como se describió en el flujo anterior.

Componentes de la Arquitectura de Onboarding

El siguiente diagrama muestra los componentes que permiten el proceso de login de un cliente del banco.



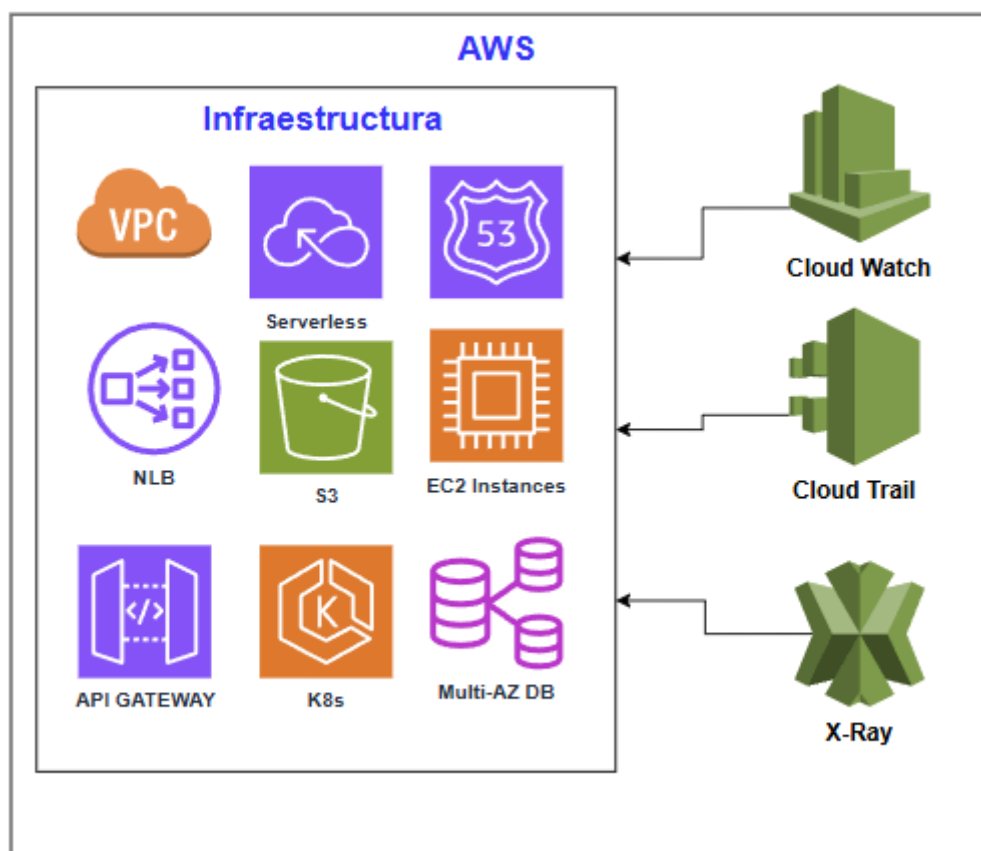
Monitoreo

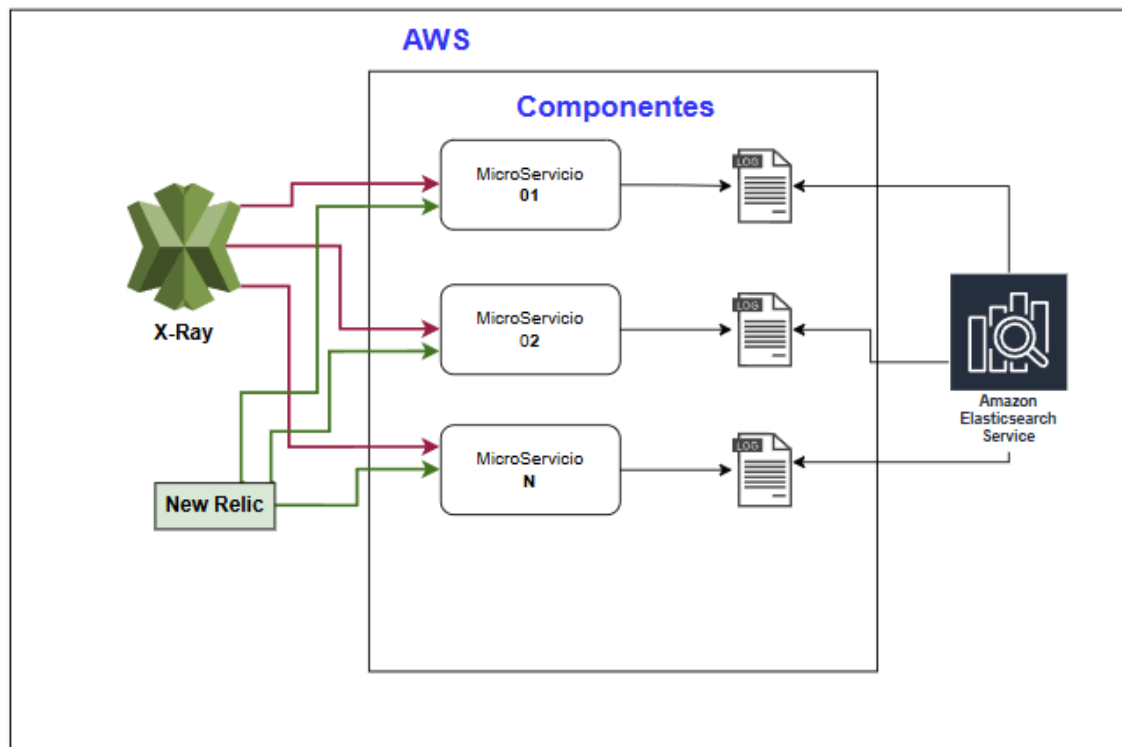
El monitoreo de un sistema bancario es crítico para garantizar la continuidad del servicio, el cumplimiento de los SLAs y las normativas que rigen a los bancos del país, por lo que se debe recopilar datos, métricas y logs de la infraestructura (hardware), los componentes de la solución (software) y eventos de seguridad de la solución.

Infraestructura: Los componentes que se encuentran en la nube deberán ser monitoreados con las herramientas propias del proveedor de nube. Por ejemplo: AWS → CloudWatch, X-Ray, Cloud Trail. Se requiere también monitorear constantemente la red, firewalls y balanceadores para garantizar una baja latencia entre las conexiones de los componentes de la solución. Se deben crear alertas automáticas para notificar de eventos en la infraestructura (altos tiempos de respuesta, desconexiones, no conexiones, etc)

Componentes de la solución: Para garantizar el buen estado de los servicios, se debe monitorear constantemente el tiempo de respuesta de cada microservicio de la solución y creando alertas automáticas cuando se superen o no se cumplan las reglas establecidas para cada caso. Entre los parámetros a analizar se encuentran: tiempos de respuesta y códigos de respuesta. Entre otras, las herramientas a utilizar serían New Relic y Cloud Trail. Además se podría usar ELK o AWS Elastic Search para revisar los logs de cada uno de los componentes de la solución.

Eventos de seguridad: Siendo la seguridad un aspecto muy importante en el sistema bancario, este aspecto se deberá cubrir haciendo uso de herramientas de software que permitan detectar vulnerabilidades en tiempo real.





Acceso a datos

El sistema bancario y los demás sistemas de apoyo deben garantizar acceso a la información con baja latencia, garantizando la integridad y seguridad de los datos (transacciones: pagos, transferencias). También deben estar disponibles según los SLAs y regulaciones de la banca local y aquellos datos más sensibles deben estar en ambientes locales (país del banco) seguros (AES-256) y bajo la custodia del banco.

DB por microservicio: Para garantizar la escalabilidad del sistema, baja latencia y disponibilidad de los datos, cada microservicio de la solución (API-Seguridad, API-OTPs, Admin. Claves, Notificaciones, Auditoría) manejará su propia base de datos lo cual permitirá mantener desacoplado el sistema, incluso a nivel de información. La base de datos se elegirá de acuerdo a las necesidades de cada componente ya que algunos de ellos (pagos, transferencias) soportarán altas cargas de escritura, mientras que otros (consultas, auditoría) tendrán intensas cargas de lectura lo cual obligará a utilizar índices, cache, denormalización de datos o incluso motores de BD no relacionales.

Cache: Aquellos componentes de la solución que requieran almacenamiento temporal de información para el normal funcionamiento del sistema, deberán hacer uso cache (Redis)

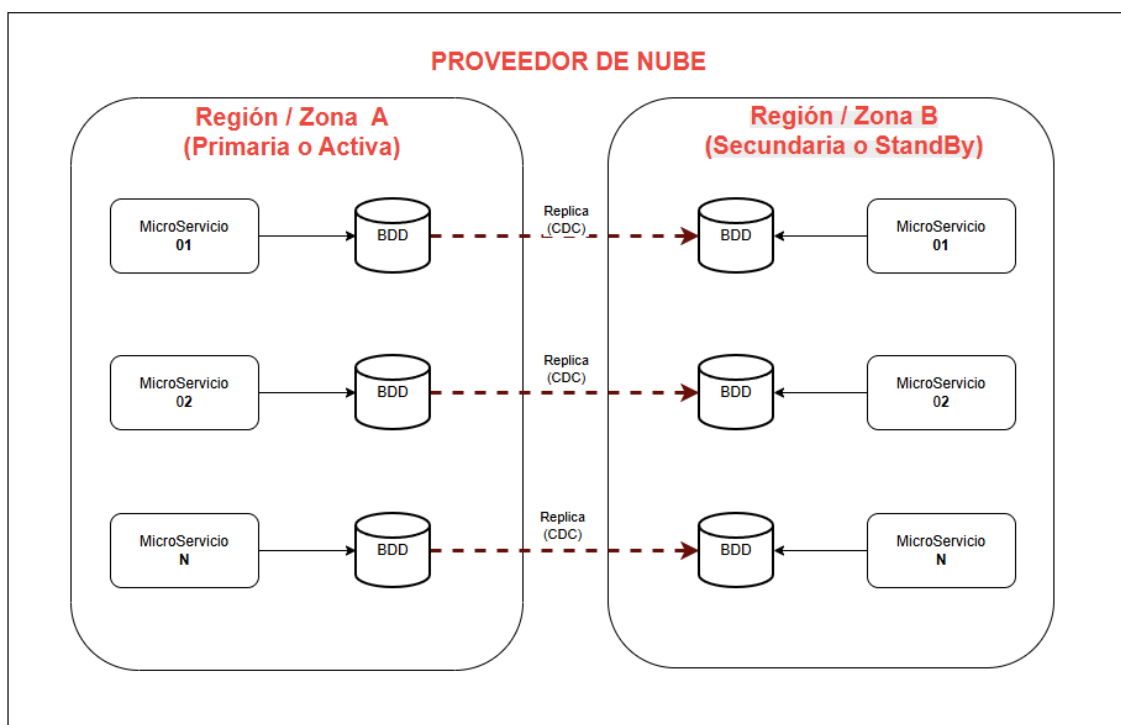
Replicación: Considerando que el sistema bancario soportará un gran volumen de transacciones por día, la BDD que almacene la información de los movimientos (pagos, transferencias) realizados deberá replicarse con el modelo MASTER-SLAVE, siguiendo el esquema CDC (Capture Data Changes) para mantener un bajo impacto en el rendimiento del sistema. Para garantizar alta disponibilidad y recuperación de un desastre, los SLAVE deben estar en lugares diferentes (diferentes proveedores de nube, diferentes regiones o diferentes zonas de disponibilidad dentro del proveedor de nube). Este esquema de replicación aplica a todas las BDD de la solución, excepto a las que se destinen al manejo de cache.

Partición de datos: La partición de datos dependerá del tipo de información que almacene la BDD. Por ejemplo para la base de datos de auditoría, la SHARD_KEY debería estar relacionada con el TIME_STAMP de cada evento registrado.

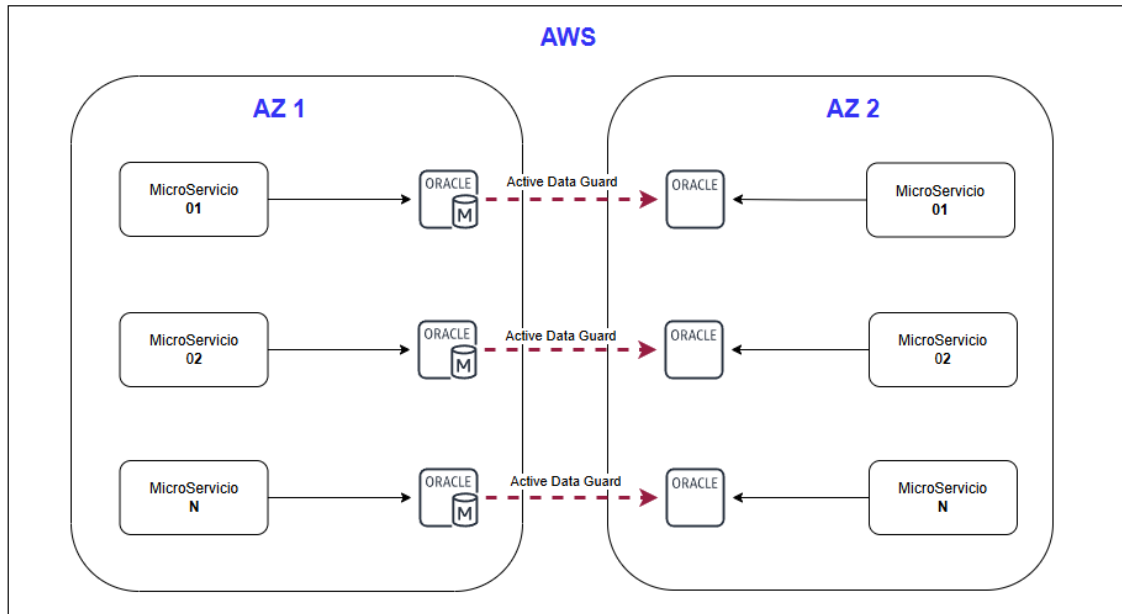
Índices: Todas las tablas de las BDD deberán tener claves primarias para cada registro y además contarán con índices que permitan recuperar los datos con la menor latencia. La mejor forma de establecer los índices es analizando los patrones de acceso a datos. Por ejemplo: La tabla que almacena los movimientos de las cuentas deberán tener un índice que facilite la lectura de las transferencias realizadas por un cliente en determinada fecha → TRX_TYPE, CLIENT_ID, TRX_DATE

Retención de datos: El tiempo de retención de datos para consulta de históricos de transacciones será de 90 días hacia atrás.

Backups: Los backups de cada base de datos se harán cada día.



Para la nube de AWS se utilizaría Amazon RDS para Oracle (v19 o v21) multi-AZ:

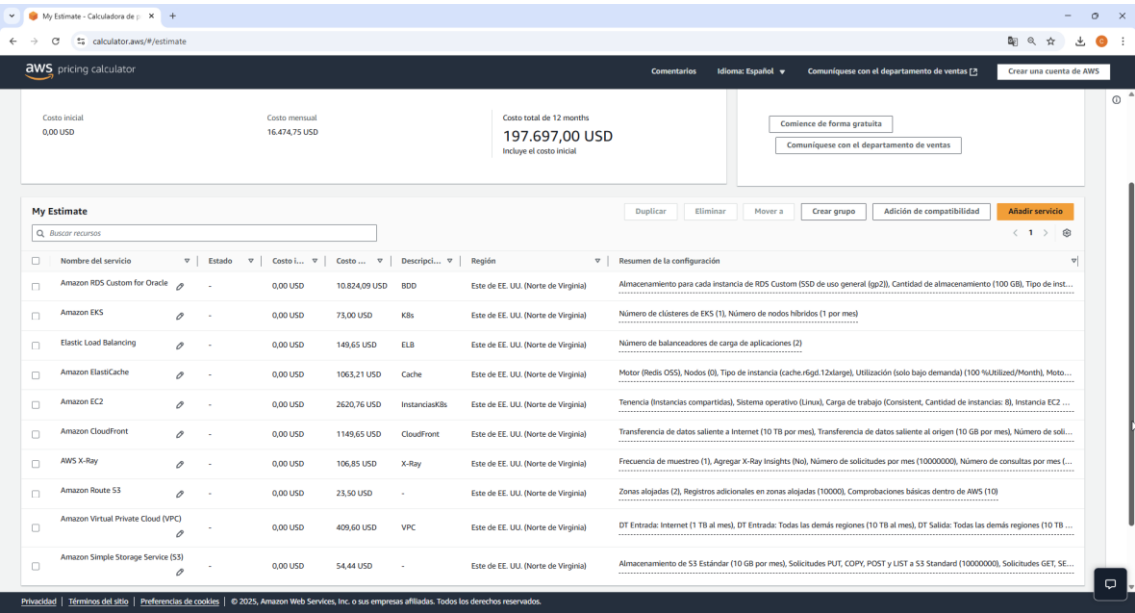


Presupuesto de la solución

Para realizar la estimación de alto nivel del presupuesto de la solución se considera lo siguiente:

- Proveedor de nube AWS
- La estimación es para el ambiente productivo
- No se consideran ambientes de INT, QA ni prePROD
- La solución se desplegará en dos regiones (Primaria: us-east-1 y Secundaria: us-west-2) para garantizar alta disponibilidad y recuperación ante desastres
- Los componentes se desplegarán en al menos 2 zonas de disponibilidad
- El auto escalado de la solución se realizará cuando se tenga: RAM>75%, CPU>70%, Latencia>200ms
- La BDD se replicará del master en la región primaria hacia el esclavo en la región secundaria
- El contenido estático se entregará desde el CloudFront para reducir la latencia y se refrescará cada 24 horas
- Todos los valores son bajo demanda

Servicio	Descripción	Costo Mensual
RDS Custom Oracle	8 Instancias primarias + 8 secundarias	\$ 10824.09
EKS	Servicio para gestionar cluste de K8s	\$ 73.00
ELB	2 Application Load Balancer	\$ 149.65
Elastic Cache	Cluster para cache en Redis	\$ 1063.21
EC2	Instancias de EC2	\$ 2620.76
X-Ray	Análisis de comportamiento de componentes	\$ 106.85
Route 53	DNS de la solución	\$ 23.50
VPCs	Redes/Sub-redes	\$ 409.60
S3	Almacen de fotos, logs, respaldos, etc	\$ 54.44
	TOTAL AL MES	\$16474.75



Resumen de arquitectura

Componente	Patrones Arquitectónicos	Uso
Todos	Clean Architecture	Principios que guían el diseño de la solución para lograr: separación de responsabilidades, escalabilidad, mantenibilidad y flexibilidad de la solución
Sistema Bancario	API Gateway	Expone un “end point” unificado para acceder a las APIs de los servicios y componentes del sistema
Sistema Bancario	Adapter	Creación de interfaces que le permitan interactuar con la “Plataforma Core” del banco u otros sistemas externos a la solución
Sistema de Auditoría	CQRS	El sistema de auditoría cuenta con un componente independiente que escribe los datos y otro que los lee y los muestra a usuarios del banco
Sistema de Auditoría	Message Broker	Para procesar la auditoría se entregan los datos a colas que son leídas por los componentes que guardan la información en BDD
Sistema Bancario Sistema de Auditoría Sistema de Notificaciones	Event-Driven Architecture	Eventos como pagos, transferencias, consultas, accesos al sistema, errores, respuestas, etc; serán producidos por los sistemas de la solución y a la vez consumidos por otros componentes para su procesamiento, logrando así bajo acoplamiento entre ellos, alto rendimiento, tolerancia a fallos
Sistema Bancario Sistema de Auditoría Sistema de Notificaciones	Microservices	Los servicios de los sistemas serán implementados en microservicios para garantizar: resiliencia, escalabilidad, flexibilidad, bajo acoplamiento
BDD	DB-per-microService	Para garantizar la escalabilidad del sistema, baja latencia y disponibilidad de los datos, cada microservicio de la solución manejará su propia base de datos lo cual permitirá mantener desacoplado el sistema, incluso a nivel de información

Con la arquitectura planteada se logra:

- Alta disponibilidad: Haciendo uso de diferentes zonas de disponibilidad de en dos regiones diferentes del proveedor de nube.
- Escalabilidad: Soportará incrementos aleatorios de tráfico creciendo/decreciendo de forma horizontal
- Bajo acoplamiento: Los componentes están diseñados para funcionar sin estar fuertemente acoplados a otros componentes.
- Tolerancia a fallos: El diseño de componentes en microservicios gestionados (K8s) permite levantar más instancias en caso de que una de ellas deje de funcionar. Por defecto el cluster de K8s se debe configurar para tener al menos 1 nodo funcionando.
- Recuperación de desastres: Las bondades de la nube permiten tener una Zona secundaria con toda la solución desplegada en "Stand by" y las bases de datos replicadas, de tal manera que en caso de desastre en la zona primaria se pueda redirigir el tráfico a la zona secundaria y garantizar la continuidad de la solución, cumpliendo con los SLAs.

